

Decidable Languages

- We use languages to represent various computational problems because we have a terminology for dealing with languages
- **Definition:** A language is *decidable* if there is an algorithm (i.e. a Turing decider) to recognize it.
- We develop examples of languages that are decidable by algorithms.

Limits of Computation – p.2/??

22c:131 Limits of Computation

Hantao Zhang

<http://www.cs.uiowa.edu/~hzhang/c131>

The University of Iowa
Department of Computer Science

Decidability

Limits of Computation – p.1/??

Theorem 4.1

A_{DFA} is a decidable language.

Proof idea: construct a TM M that decides A_{DFA}
 $M =$ "On input $\langle B, w \rangle$, where B is a DFA and w is a string

1. Simulate B on w
2. If the simulation ends in an accept state then *accept*; if it ends in a nonaccepting state then *reject*."

Note: w is finite and simulation always ends

Limits of Computation – p.4/??

Membership Problem for DFA

- Consider the acceptance problem for DFAs: *test whether a particular finite automaton accepts a given string*.
This can be expressed as a language A_{DFA}
- A_{DFA} contains the encodings of all DFAs together with strings the DFAs accept, i.e.. $A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts } w \}$
- Hence, testing whether DFA B accepts w is the same as testing whether $\langle B, w \rangle \in A_{DFA}$
- Other computational problems are formulated in terms of testing membership in a language.
- To show that a computational problem is decidable is to show that the encoding of the problem is decidable.

Limits of Computation – p.3/??

Acceptance Problem for NFA

$A_{NFA} = \{\langle B, w \rangle \mid B \text{ is an NFA and } w \text{ is a string}\}$

Theorem A_{NFA} is a decidable language

Proof: Construct a TM N that decides A_{NFA} .

Because M is designed to work with *DFAs*, N first converts its input *NFA* to a *DFA* by the usual technique

$N =$ "On input $\langle B, w \rangle$ where B is an NFA and w is a string

1. Convert NFA B to a DFA C (see Theorem 1.19)
2. Run TM M from Theorem 4.1 on $\langle C, w \rangle$
3. If M accepts, *accept*; otherwise *reject*"

Note: running M in stage 2 means incorporating M into the design of N as a subprocedure

Limits of Computation – p.6/??

Performing the Simulation

- $\langle B, w \rangle$ is a representation of a DFA B together with a string w . One can represent B by a list of its five components: $(Q, \Sigma, \delta, q_0, F)$
- When M receives an input it checks first whether this input represents a DFA B and a string w ; if not reject
- If input is right, M keeps track of B 's current state and B 's current position in w by writing this info on its tape
- Initially the state of B is q_0 and B 's current position is the leftmost symbol of w ; the states and position are updated as shown by δ
- When M finishes processing the last symbol of w , M accepts if B is in a final state and reject if B is not in a final state

Limits of Computation – p.5/??

Theorem 4.4

E_{DFA} is a decidable language

Proof idea:

- A DFA accepts some string iff reaching a final state from the start state by traveling along the arrows of the DFA is possible.
- To test this condition we can construct the TM T that marks states of DFA using the state transition function of the DFA
- Use T to solve emptiness problem

Limits of Computation – p.8/??

Emptiness Problem

- Another kind of problems concerning FAs is the *emptiness testing*
- **The problem:** test if the language of a DFA is empty
- For this case we consider the language $E_{DFA} = \{\langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset\}$

Limits of Computation – p.7/??

Language Equality

- **The problem:** for two DFA-s A and B , is $L(A) = L(B)$?
- **The language:** $EQ_{DFA} = \{\langle A, B \rangle \mid A, B \text{ are DFAs, and } L(A) = L(B)\}$

Limits of Computation – p.10/??

The TM T

T = "On input $\langle A \rangle$ where A is a DFA:

1. Mark the start state of A
2. Repeat until no new states get marked:
3. Mark any state that has a transition coming into it from any state that is already marked
4. If no final state is marked, *accept*; otherwise *reject*"

Limits of Computation – p.9/??

Theorem 4.5

EQ_{DFA} is a decidable language

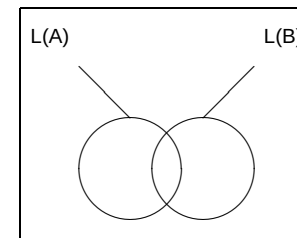
Proof idea: (use Theorem 4.4)

- Construct a DFA C from A and B where C accepts only those strings that are accepted either by A or B but not by both.
- If A and B recognize the same language then C accepts nothing
- The language C is defined by
$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$
which is called *symmetric difference* of $L(A)$ and $L(B)$
- Use machine T to check if C is empty

Limits of Computation – p.12/??

Symmetric Difference

The expression $L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$ called the symmetric difference of $L(A)$ and $L(B)$ is illustrated here:



Limits of Computation – p.11/??

$L(A) = \Sigma^*$ for DFA A ?

- **Problem:** Given any DFA A on Σ a finite alphabet, can we decide $L(A) = \Sigma^*$?
- **Language:** $ALL_{DFA} = \{\langle A \rangle \mid A \text{ is a DFA that recognize } \Sigma^*\}$
- Show that ALL_{DFA} is decidable

Proving Theorem 4.5

Construct the TM F :

F = "On input $\langle A, B \rangle$ where A and B are DFA:

1. Construct DFA C that recognizes $L(C)$ as described above
 $L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$
2. Run TM T from Theorem 4.4 on input $\langle C \rangle$
3. If T accepts, *accept*; if T rejects, *reject*."

Decision Problems of CFLs

- Describe algorithms to test whether a CFG generates a particular string.
- Describe algorithms to test whether the language generated by a CFG is empty.
- The language we consider is:
 $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG that generates string } w\}$

Solution

The TM L that decides ALL_{DFA} uses the fact that $\overline{L(A)}$ is regular

L = "On input $\langle A \rangle$ where A is a DFA:

1. Construct DFA B that recognizes $\overline{L(A)}$ by swapping accept and unaccept states in A
2. Run the TM T that decides the emptiness E_{DFA} on B
3. If T accepts, *reject*; if T rejects *accept*."

A better idea

- Make the recognizer a decider. For that we need to ensure that the algorithm tries only finitely many derivations

Theorem 4.7

A_{CFG} is a decidable language

Proof ideas: For a CFG G and a string w :

- **First idea:** Go through all derivations generated by G checking whether any is a derivation of w .
Since there are infinitely many derivations this idea does not work. If G does not generate w the algorithm doesn't halt. I.e, this idea provides a recognizer but not a decider

Checking CFG derivations

- Convert G into Chomsky normal form
- For a string w of length $|w| = n$ check all derivations with $2n - 1$ steps to determine whether G generates w
- Since we can convert G into a Chomsky normal form (see Section 2.1), this is a good idea

Fact

If G is a CFG in Chomsky normal then for any $w \in L(G)$ where $|w| = n$ exactly $2n - 1$ steps are required for any derivation of w

Proof: Induction on n .

Emptiness testing for CFG

Consider the language

$$E_{CFG} = \{\langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset\}$$

Theorem 4.7: E_{CFG} is a decidable language

Proof of Theorem 4.7

The TM S that decide A_{CFG} is:

S = "On input $\langle G, w \rangle$, where G is a CFG and w is a string:

1. Convert G to an equivalent grammar in Chomsky normal form
2. List all derivations with $2n - 1$ steps, $n = \text{length}(w)$ except if $n = 0$; in this case list all derivations with 1 step
3. If any of the derivations listed above generates w , *accept*; if not *reject*."

Proof of Theorem 4.8

Construct the TM R :

R = "On input $\langle G \rangle$ where G is a CFG:

1. Mark all terminal symbols of G
2. Repeat until no new variable get marked:
Mark any variable A where G has a rule $A \rightarrow u_1 u_2 \dots u_k$ and each symbol $u_1, u_2, \dots, u_k$ has already been marked
3. If the start symbol of G is not marked, *accept*; otherwise *reject*.

Proof idea

- To test whether $L(G)$ is empty we need to test whether the axiom of G can generate a string of terminals
- We may solve however a more general problem, determining *for each variable* whether that variable can generate a string of terminals
- When the algorithm determines that a variable can generate a string of terminals the algorithm mark that variable
- The algorithm start by marking first all terminals, Then it marks variables that have on their rhs in some rules only terminals, i.e., marked symbols, and so one

Fact 1

Class of CF languages is not closed under intersection.

Proof: By construction.

- Consider the CF languages $A = \{a^m b^n c^n \mid m, n \geq 0\}$ and, $B = \{a^n b^n c^m \mid m, n \geq 0\}$ generated by the grammars:
 $S \rightarrow RT, R \rightarrow aR \mid \epsilon, T \rightarrow bTc \mid \epsilon$ and
 $S \rightarrow TR, T \rightarrow aTb \mid \epsilon, R \rightarrow cR \mid \epsilon$ respectively.
- $L(A) \cap L(B) = \{a^n b^n c^n \mid n \geq 0\}$ which is not a CFL
- This establishes Fact 1.

Theorem 4.9

Every context-free language is decidable.

Proof: Let G be a CFG for A , i.e. $L(G) = A$. Design a TM M_G that decides A by building a copy of G into M_G :

$M_G =$ "On input w :

- Run TM S on input $\langle G, w \rangle$
- If this machine accepts, *accept*; if it rejects, *rejects*

Note: TM S converts G to Chomsky normal form, and produces all derivations of length $2n - 1$ where $n = |w|$. Then check if w is among the derived strings.

CFL equality problem

Consider the language:

$$EQ_{CFG} = \{\langle G, H \rangle \mid G, H \text{ are CFG and } L(G) = L(H)\}$$

Note:

- Since class of CF languages is not closed under intersection and complementation (as seen before), we cannot use the symmetric difference for EQ_{CFG} .
- In fact EQ_{CFG} is not decidable (to be proven later)

Fact 2

Class of CF languages is not closed under complementation.

Proof: By contradiction. Assume that CFL is closed under complementation

- If G_1 and G_2 are two CFG then $\overline{L(G_1)}$ and $\overline{L(G_2)}$ are CFL
- Then $\overline{L(G_1)} \cup \overline{L(G_2)}$ is a CFL. Hence, $\overline{\overline{L(G_1)} \cup \overline{L(G_2)}}$ is a CFL.
- By DeMorgan's law $\overline{\overline{L(G_1)} \cup \overline{L(G_2)}} = L(G_1) \cap L(G_2)$, a contradiction because class of CFL is not closed under intersections.

Example 1

Equivalence of DFA and REX:

- Consider the problem of testing whether a DFA and a regular expression are equivalent.
- Express this problem as a language and show that this language is decidable

Methodology

To solve decidability problems concerning relations between languages one should proceed as follows:

- Understand the relationship
- Transform the relationship into an expression using closure operators on decidable languages
- Design a TM that construct the language thus expressed
- Run a TM that decide the language represented by the expression

Example 2

Decidability of Σ^*

- **Problem:** is the language Σ^* , for Σ a finite alphabet, decidable?
- **Language:** $ALL_{DFA} = \{\langle A \rangle \mid A \text{ is a DFA that recognize } \Sigma^*\}$ Show that ALL_{DFA} is decidable

Solution

- Let $EQ_{DFA,REX} = \{\langle A, R \rangle \mid A \text{ is a DFA, } R \text{ is a regular expression and } L(A) = L(R)\}$
- The following TM E decides $EQ_{DFA,REX}$:
 $E =$ "On input $\langle A, R \rangle$
 1. Convert R to an equivalent DFA B
 2. Use the TM C for deciding EQ_{DFA} on input $\langle A, B \rangle$
 3. If C accepts, *accept*; if C rejects, *reject*."

Note: C constructs the symmetric difference $(L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$ and test if it is empty

Example 3

Using CFG and REX

- **Problem:** show that the problem of testing whether a CFG generates some string in 1^* is decidable.
- **Language:** $A = \{\langle G \rangle \mid G \text{ is a CFG over } \{0, 1\}^* \text{ and } 1^* \cap L(G) \neq \emptyset\}$

Solution

The TM L that decides ALL_{DFA} uses the fact that $\overline{L(A)}$ is regular

$L =$ "On input $\langle A \rangle$ where A is a DFA:

1. Construct DFA B that recognizes $\overline{L(A)}$ by swapping accept and unaccept states in A
2. Run the TM T that decides the emptiness E_{DFA} on B
3. If T accepts, *reject*; if T rejects *accept*."

Example 4

Example regular expressions

- **Problem:** Is the language generated by a particular regular expression decidable? For example, is the language of regular expressions that contain at least one string that has the pattern "111" as a substring decidable?
- **Language:** $A = \{\langle R \rangle \mid R \text{ is a regular expression describing a language that contain at least one string } w \text{ that has "111" as a substring (i.e., } w = x111y \text{ where } x \text{ and } y \text{ are strings)}\}$

Solution

Assume that G is over $\{0, 1\}^*$. Then we need to show that the language $A = \{\langle G \rangle \mid G \text{ is a CFG over } \{0, 1\}^* \text{ and } 1^* \cap L(G) \neq \emptyset\}$ is decidable. Since 1^* is regular and $L(G)$ is CFL then $1^* \cap L(G)$ is a CFL. Hence the TM X that decides A is:

$X =$ "On input $\langle G \rangle$ where G is a CFG:

1. Construct CFG H such that $L(H) = 1^* \cap L(G)$
2. Run the TM R that decides the language E_{CFG} on $\langle H \rangle$
3. If R accepts, *reject*; if R rejects, *accept*."

Halting Problem of TM

- It is also the membership problem of TM: whether a Turing machine accepts a given input string.
- By analogy with A_{DFA} and A_{CFG} we call the corresponding language A_{TM}
 $A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$
- Contrasting A_{DFA} and A_{CFG} , which are decidable, A_{TM} is not decidable.

Solution

The language A is decidable. The reason is that language A can be expressed using regular operators as $L(\Sigma^* \cdot 111 \cdot \Sigma^*) \cap L(R)$. Hence, the TM X that decides A is:

$X = \text{"On input } \langle R \rangle \text{ where } R \text{ is a regular expression:}"$

1. Construct the DFA E that accepts $\Sigma^*111\Sigma^*$
2. Construct the DFA B that accepts $L(B) = L(R) \cap L(E)$
3. Run TM T that decide E_{DFA} on input $\langle B \rangle$
4. If T rejects *accept*; if T accepts *reject*."

A recognizer for A_{TM}

The following TM U recognizes A_{TM}

$U = \text{"On input } \langle M, w \rangle, \text{ where } M \text{ is a TM and } w \text{ is a string}"$

1. Simulate M on input w
2. If M ever enters its accept state, *accept*; if M ever enters its reject state, *reject*".

Theorem 4.11

A_{TM} is undecidable.

- A_{TM} is however Turing-recognizable
- Hence, Theorem 4.11 shows that recognizers are more powerful than deciders.
- Requiring a TM to halt on all inputs restricts the kind of languages that it can recognize

Observations

- The TM U is interesting in its own right because it is an example of the *universal Turing machine*, first proposed by Turing
- U is called universal because it is capable to simulate any other Turing machine from the description of that machine
- The universal TM played an important role in the stimulating the development of stored-program computers

Note

- U loops on the input $\langle M, w \rangle$ if M loops on w . This is why U does not decide A_{TM} .
- If the algorithm had some way to determine that M was not halting on w , it could reject. This is why it is called the halting problem.
- However, as we demonstrate Theorem 4.11, an algorithm has no way to make this determination

Example Infinite Sets

- The set of strings over $\{0, 1\}$ is an infinite set
- The set $\mathcal{N}$ of natural numbers is also an infinite set
- The set $\mathcal{E}$ of all even natural numbers is also an infinite set
- How can we compare them?

How to Prove Undecidability

- The proof of undecidability of the TM membership problem uses Georg Cantor (1873) technique called *diagonalization*.
- Cantor's problem was to measure the size of infinite sets.
- The size of finite sets is measured by counting the number of their elements.
- The size of infinite sets cannot be measured by counting their elements because this procedure does not halt.

The Correspondence

Consider two sets, A and B and $f : A \rightarrow B$ a function

- f is *one-to-one* if it never maps two different elements of A into the same element of B , i.e., $f(a) \neq f(b)$ whenever $a \neq b$.
- f is *onto* if it hits every element of B , i.e., $\forall b \in B, \exists a \in A$ such that $f(a) = b$
- f is called a *correspondence* if it is both *one-to-one* and *onto*

Two sets A and B have the same size if there is a correspondence $f : A \rightarrow B$

Cantor's solution

- Two finite sets have the same size if their elements can be paired
- Since this method does not rely on counting elements it can be used for both finite and infinite sets

Definition 4.14

A set is *countable* if either it is finite or it has the same size as $\mathcal{N}$.

Example Correspondences

- Let $\mathcal{N}$ be the set of natural numbers, $\mathcal{N} = \{1, 2, 3, \dots\}$ and $\mathcal{E}$ the set of even natural numbers, $\mathcal{E} = \{2, 4, 6, \dots\}$
- Intuitively one may believe that $size(\mathcal{N}) > size(\mathcal{E})$ because $\mathcal{E} \subset \mathcal{N}$ but not the opposite. However, using Cantor's method we can show that $\mathcal{N}$ and $\mathcal{E}$ have the same size by constructing the correspondence $f : \mathcal{N} \rightarrow \mathcal{E}$ defined by $f(n) = 2n$.

Correspondence $\mathcal{Q} \leftrightarrow \mathcal{N}$

1. Put $\mathcal{N}$ on two axes
2. Line i contains all rationals that have numerator i , i.e. $\{\frac{i}{j} \in \mathcal{Q} \mid i \in \mathcal{N} \text{ fixed}, \forall j \in \mathcal{N}\}$
3. Column j contains all rationals that have denominator j , i.e. $\{\frac{i}{j} \in \mathcal{Q} \mid \forall i \in \mathcal{N}, j \in \mathcal{N} \text{ fixed}\}$
4. Number $\frac{i}{j}$ occurs in i -th row and j -th column

Limits of Computation – p.50/??

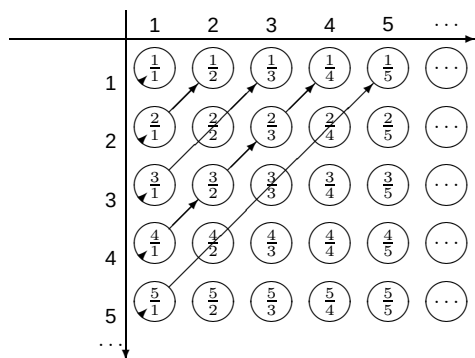
A complex correspondence

Let $\mathcal{Q}$ be the set of positive rational numbers,
 $\mathcal{Q} = \{\frac{m}{n} \mid m, n \in \mathcal{N}\}$

- Intuitively, $\mathcal{Q}$ seems to be much larger than $\mathcal{N}$
- Yet we can show that this two sets have the same size by constructing the correspondence in the next slide.

Limits of Computation – p.49/??

The list of rational numbers



Limits of Computation – p.52/??

Turning $\{\frac{i}{j} \mid i, j \in \mathcal{N}\}$ into a list

- Bad idea: list first elements of a line or a column. Lines and columns are labeled by $\mathcal{N}$, hence this would never end
- Good idea (Cantor's idea): use the diagonals:
 1. First diagonal contains $\frac{1}{1}$, i.e, first element of the list is $\frac{1}{1}$
 2. Continue the list with the elements of the next diagonal: $\frac{2}{1}, \frac{1}{2}$
 3. Continue this way skipping the elements that may generate repetitions, such $\frac{i}{i}$ that would generate a copy of $\frac{1}{1}$.

Limits of Computation – p.51/??

Uncountable sets

An infinite set for which no correspondence with $\mathcal{N}$ can be established is called *uncountable*

Example of uncountable set: the set $\mathcal{R}$ of real numbers is uncountable

Proof: Cantor proved that $\mathcal{R}$ is uncountable using the diagonalization method

More Infinite Countable Sets

- The set $\mathcal{N} \times \mathcal{N}$
- The set $\mathcal{N}^k$ for any $k \in \mathcal{N}$
- The set Σ^*
- Any subset of a countable set is also countable.

Construction

- Since $f : \mathcal{N} \rightarrow \mathcal{R}$ is a correspondence $\mathcal{R}$ can be listed as seen here:

n	$f(n)$
1	3.14159...
2	55.5555...
3	0.1234...
4	0.5000...
...	...

Theorem 4.17

$\mathcal{R}$ is uncountable.

Proof: We will show that no correspondence exist between $\mathcal{N}$ and $\mathcal{R}$.

- Suppose that such a correspondence $f : \mathcal{N} \rightarrow \mathcal{R}$ exists and deduce a contradiction showing that f fail to work properly.
- We construct an $x \in \mathcal{R}$ that cannot be the image of any $n \in \mathcal{N}$.

Application

Theorem 4.17 shows that some languages are not decidable or even Turing recognizable.

Reason:

- There are uncountable many languages yet only countable many Turing machines.
- Because each Turing machine can recognize a single language and there are more languages than Turing machines some languages are not recognized by any Turing machine
- Such languages are not Turing recognizable

Formal construction of x

Construct $x \in (0, 1)$ by the following procedure:

- $x = 0.d_1d_2d_3d_4 \dots$ has an infinite number of decimals constructed by the rule:
 $\forall i \in \mathcal{N}$ chose d_i a digit different from the i -th digit of $f(i)$
- Consequence: $\forall i \in \mathcal{N}, x \neq f(i)$. Hence, x does not belong to the list $\mathcal{R}$ and thus f is not a correspondence.

Uncountable Sets

- The real numbers $\mathcal{R}_1 = (0, 1)$
- The infinite-length binary strings $\mathcal{B}$
- The integer functions $\mathcal{F} = \{f \mid f : \mathcal{N} \rightarrow \{0, 1\}\}$
- The powerset $\mathcal{P}(\mathcal{N})$
- The set of all formal languages $\mathcal{L} = \{L \mid L \subseteq \Sigma^*\}$

Countable Sets

- Any finite set
- Natural numbers $\mathcal{N}$
- Rational numbers $\mathcal{Q}$
- Any subset of a countable set
- Cartesian product of countable sets
- Σ^*

$\mathcal{L}$ is uncountable

- The set of all formal languages $\mathcal{L} = \{L \mid L \subseteq \Sigma^*\}$
- We will show that $\mathcal{L}$ is uncountable by constructing a correspondence $\mathcal{B} \rightarrow \mathcal{L}$.
- Since Σ^* is countable, $\Sigma^* = \{s_1, s_2, s_3, \dots\}$
- Each language $A \in \mathcal{L}$ has a unique infinite binary sequence $\chi_A \in \mathcal{B}$ constructed by:
the i -th bit of χ_A , $\chi_A(i) = 1$ if $s_i \in A$ and $\chi_A(i) = 0$ if $s_i \notin A$.
- χ_A is the characteristic function of A in Σ^*
- The function $f : \mathcal{L} \rightarrow \mathcal{B}$ where $f(A) = \chi_A$ is one-to-one and onto and hence it is a correspondence.

Limits of Computation – p.62/??

A characteristic function

Let $\Sigma = \{0, 1\}$ and A the language of all strings starting with 0 over Σ .

- $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 11, 000, 001, 010, 011, 100, \dots\}$;
- $A = \{0, 00, 01, 000, 001, 010, 011, \dots\}$;
- $\chi_A = 0\ 1\ 0\ 1\ 1\ 0\ 1\ 1\ 1\ 1\ \dots$

Limits of Computation – p.61/??

Halting Problem is Undecidable

We are ready to prove that the language $A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ is undecidable.

Proof Idea: By contradiction, assuming that A_{TM} is decidable.

- Suppose that H is a decider of A_{TM} .
- On input $\langle M, w \rangle$ where M is a TM and w is a string, H halts and accepts if M accepts w .
- Furthermore, H halts and reject if M fails to accept w .

Limits of Computation – p.64/??

Corollary 4.18

Some languages are not Turing-recognizable.

Proof idea:

- The set of all languages is uncountable.
- The set of all Turing machines is countable.

Limits of Computation – p.63/??

Proof, continuation

Construct a new TM D that uses H as a subroutine.

- D calls H to determine what M does when its input is $w = \langle M \rangle$
- If M accepts $\langle M \rangle$ then D rejects;
if M rejects $\langle M \rangle$ then D accepts

Equational expression of H

$$H(\langle M, w \rangle) = \begin{cases} \text{accept}, & \text{if } M \text{ accepts } w; \\ \text{reject}, & \text{if } M \text{ does not accept } w. \end{cases}$$

A CONTRADICTION

$$D(\langle M \rangle) = \begin{cases} \text{accept}, & \text{if } M \text{ does not accept } \langle M \rangle; \\ \text{reject}, & \text{if } M \text{ accepts } \langle M \rangle. \end{cases}$$

What happens when we ran D on $\langle D \rangle$?

$$D(\langle D \rangle) = \begin{cases} \text{accept}, & \text{if } D \text{ does not accept } \langle D \rangle; \\ \text{reject}, & \text{if } D \text{ does not reject } \langle D \rangle. \end{cases}$$

This is a contradiction and consequently neither TM D nor TM H do exist.

The machine D

D = "On input $\langle M \rangle$, where M is a TM:

1. Run H on input $\langle M, \langle M \rangle \rangle$
2. Output the opposite of what H outputs:
accepts if H rejects and *rejects* if H accepts."

Where is diagonalization?

To make the use of diagonalization obvious we construct the list of all Turing machines running on Turing machines as input here:

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	accept		accept		
M_2	accept	accept	accept	accept	...
M_3					...
M_4	accept	accept			

Summarizing

- Assume that H decides A_{TM}
 - Use H to build D that accepts $\langle M \rangle$ when M rejects and rejects $\langle M \rangle$ when M accepts
 - H and D performs as follows:
 - H accepts $\langle M, w \rangle$ exactly when M accepts w
 - D rejects $\langle M \rangle$ exactly when M accepts $\langle M \rangle$
 - D rejects $\langle D \rangle$ exactly when D accepts $\langle D \rangle$
- This is a contradiction and neither H nor D can exist

Running D on $\langle D \rangle$

The result of running H on the machine in the previous slide when D is present.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	accept	reject	accept	reject	...	accept	...
M_2	accept	accept	accept	accept	...	accept	...
M_3	reject	reject	reject	reject	...	reject	...
M_4	accept	accept	reject	reject	...	accept	...
D	reject	reject	accept	accept	...	???	...

Running H

The result of running H on the machine in the previous slide.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	accept	reject	accept	reject	...
M_2	accept	accept	accept	accept	...
M_3	reject	reject	reject	reject	...
M_4	accept	accept	reject	reject	...

A New Concept

Co-Turing recognizable languages

- Complement of a language A , $\mathcal{C}(A)$, is the language consisting of all strings that does not belong to A .
- A language is co-Turing-recognizable if it is the complement of a Turing-recognizable language

Note

We can construct a Turing-unrecognizable language

- A_{TM} is an example of Turing undecidable language. But it is Turing recognizable
- Now we construct a language which is Turing-unrecognizable.
- This construction relies on the fact that if both a language and its complement are Turing-recognizable the language is decidable
- Hence: *for any undecidable language, either the language or its complement is not Turing-recognizable*

Proof

if Assume that A is decidable. Since complement of a decidable language is decidable it result that both A and $\mathcal{C}(A)$ are Turing-recognizable.

only if Assume that both A and $\mathcal{C}(A)$ are Turing-recognizable. Let M_1 be a recognizer for A and M_2 a recognizer for $\mathcal{C}(A)$. Then the following TM M is a decider for A

Theorem 4.16

A language is decidable iff it is both Turing-recognizable and co-Turing recognizable

I.e., a language A is decidable iff both A and $\mathcal{C}(A)$ are Turing-recognizable

Note

- Running two machines M_1 and M_2 by a machine M in parallel means that M has two tapes, one for simulating M_1 and other for simulating M_2
- M takes turns, simulating one step of each machine, which continues until one of the machines halts.
- Because $w \in A$ or $w \in \mathcal{C}(A)$ either M_1 or M_2 must accept w .
- Because M halts whenever M_1 or M_2 accepts, M always halts, so it is a decider. Further, it accepts all strings from A and rejects all strings not in A .

Construction

M = "On input w :

1. Run both M_1 and M_2 on w in parallel
2. If M_1 accepts w *accept*; if M_2 accepts w *reject*."

Corollary

$\mathcal{C}(A_{TM})$ is not Turing-recognizable

Proof: We know that A_{TM} is Turing-recognizable. If $\mathcal{C}(A_{TM})$ also were Turing-recognizable then A_{TM} would be decidable. But we have proved (Theorem 4.11) that A_{TM} is not decidable. Hence, $\mathcal{C}(A_{TM})$ must not be Turing-recognizable.

Conclusion

M is a decider for A , thus A is decidable